

I2236 INTRODUCTION TO EMBEDDED SYSTEMS

LECTURE 2 ARDUINO ECOSYSTEM & PROGRAMING BASICS

Bilal Daass
04/02/2026



COURSE OVERVIEW

Module 1: Introduction & Foundations (Week 1–2)

Module 2: Arduino Ecosystem & Programming Basics (Week 3–4)

Module 3: Digital & Analog Interfacing (Week 5–6)

Module 4: Interfacing Peripherals (Week 7–8)

Module 5: Sensors & Actuators (Week 9–10)

Module 6: Communication & Integration (Week 11–12)

Module 7: Project (Week 13–15)

MODULE 2 : ARDUINO ECOSYSTEM & PROGRAMING BASICS

- **Arduino boards: categories, specifications, schematics.**
- **Review of C programming: syntax, variables, operators, control statements.**
- **Embedded toolchain: Arduino IDE, simulator setup.**
- **Writing basic Arduino programs and testing.**

MODULE 2 : ARDUINO ECOSYSTEM & PROGRAMING BASICS

- **Arduino boards: categories, specifications, schematics.**
- Review of C programming: syntax, variables, operators, control statements.
- Embedded toolchain: Arduino IDE, simulator setup.
- Writing basic Arduino programs and testing.

MODULE 2 : ARDUINO ECOSYSTEM & PROGRAMING BASICS

- Arduino boards: categories, specifications, schematics.
- **Review of C programming: syntax, variables, operators, control statements.**
- Embedded toolchain: Arduino IDE, simulator setup.
- Writing basic Arduino programs and testing.

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

WHAT IS ARDUINO?

Arduino is an open-source platform used for building electronic projects

Arduino board is able to

- read inputs
 - temperature on a sensor,
 - a finger on a button,
 - ...
- generate output
 - activating a motor,
 - turning on an LED
 - ...

You can tell your board what to do by programming the microcontroller on the board

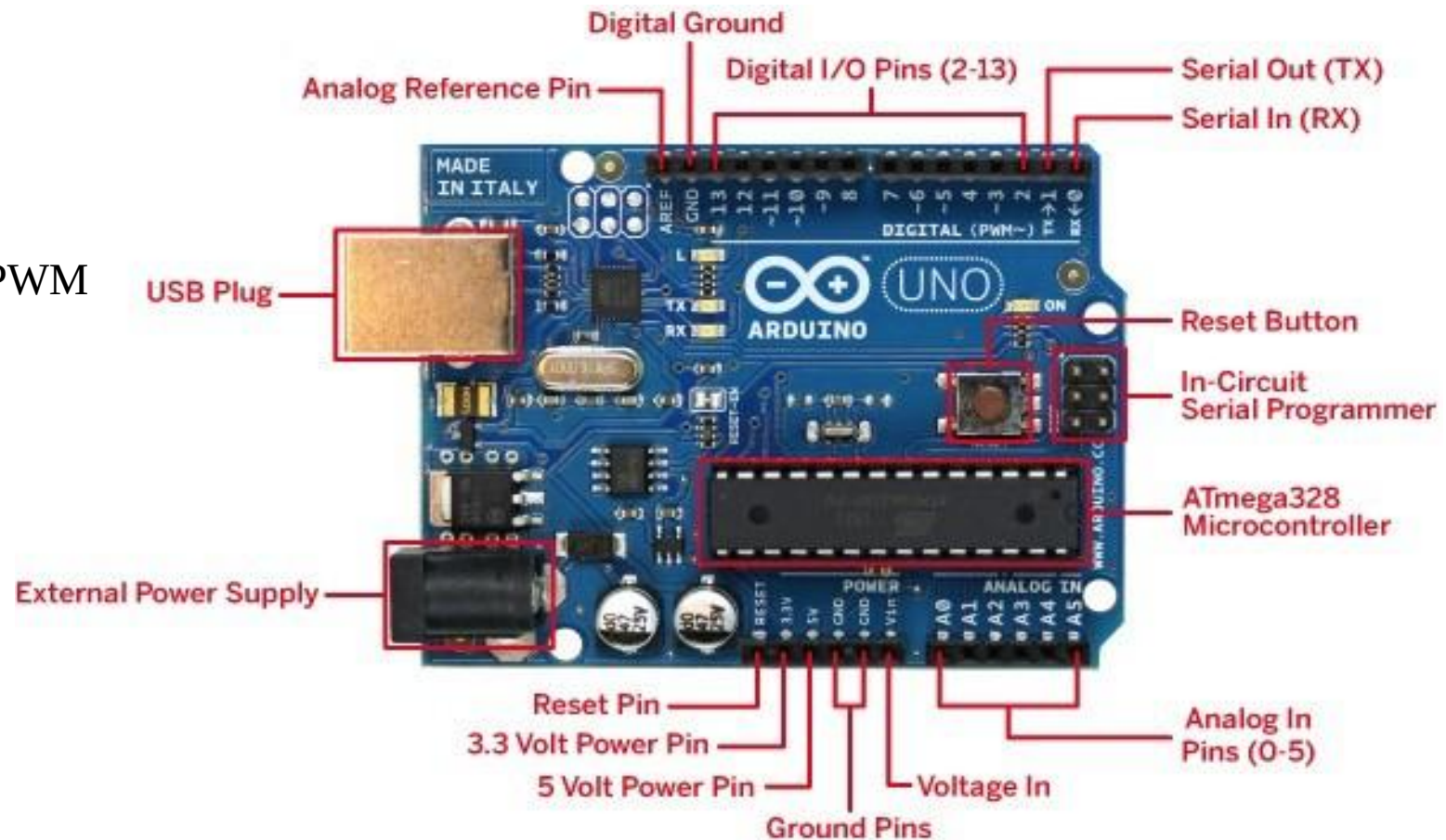


Physical programmable circuit board

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

ARDUINO UNO LAYOUT

- Microcontroller ATmega328
- Input Voltage (recommended) 7-12V
- Input Voltage (limits) 6-20V
- Digital I/O Pins 14 (of which 6 provide PWM output)
- Analog Input Pins 6
- DC Current per I/O Pin 20 mA
- DC Current for 3.3V Pin 50 mA
- Flash Memory 32 KB (ATmega328)
- SRAM 2 KB (ATmega328)
- EEPROM 1 KB (ATmega328)
- Clock Speed 16 MHz



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

Arduino Board	Processor	Memory	Digital I/O	Analog I/O
Arduino Uno / Nano	16Mhz ATmega328	2KB SRAM, 32KB flash	14	6 input, 0 output
Arduino Leonardo / Micro	16MHz ATmega32u4	2.5KB SRAM, 32KB flash	20	12 input, 0 output
Arduino 101	32Mhz Intel Curie	24KB SRAM, 196KB flash	14	6 input, 0 output
Mega 2560	16Mhz ATmega2560	8KB SRAM, 256KB flash	54	16 input, 0 output
Zero	48MHz ATSAMD21G18	32KB SRAM, 256KB flash	14	6 input, 0 output
Arduino DUE	84Mhz ATSAM3X8E	96KB SRAM, 512KB flash	54	12 input, 0 output

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

ARDUINO KIT EXAMPLE



Package Included:

- 1 x UNO Board
- 1 x USB Cable
- 1 x Jump Cable
- 1 x Breadboard
- 5 x LED Light
- 1 Pack Resistor
- 1 x Female to male dupont line
- 1 x Potentiometer
- 1 x Buzzer
- 1 x 74HC595
- 1 x Infrared receiver
- 1 x ds18b20
- 1 x Flame Sensor
- 1 x Ball Switch
- 1 x Photoresistor
- 1 x Key button
- 1 x Remote control
- 1 x 4-digit display tube
- 1 x 8*8 Dot matrix module
- 1 x 1-digit display tube
- 1 x Stepper motor driver board
- 1 x Stepper motor
- 1 x 9g Servo
- 1 x IIC 1602 LCD
- 1 x XY joystick module
- 1 x Temperature module
- 1 x Water test module
- 1 x RFID Module
- 1 x RFID keychain
- 1 x RFID White card
- 1 x Sound Module
- 1 x Relay Module
- 1 x Clock Module
- 1 x 4*4 Key board
- 1 x RGB 3 color module
- 1 x 9V battery Snap

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

ARDUINO KIT EXAMPLE



A couple of electronics suppliers that we have used in the past are:

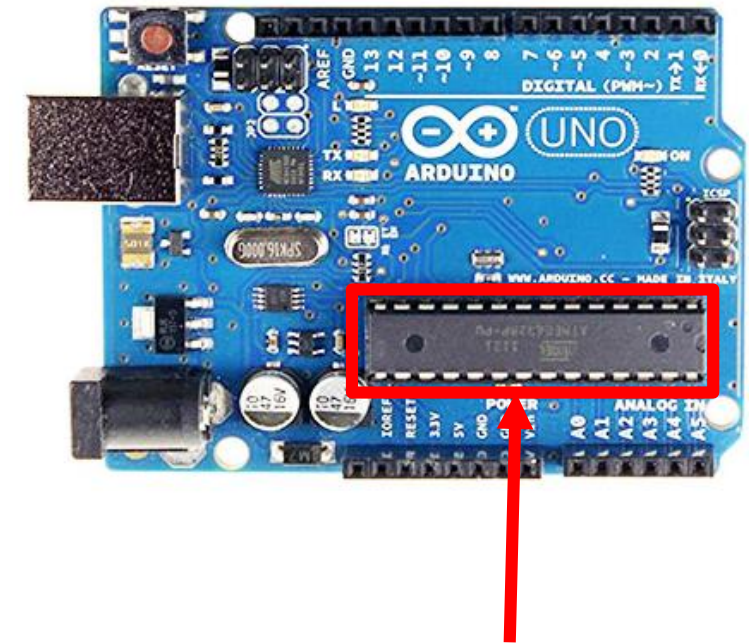
- DIYelectic : <http://www.diyelectronicslb.com/>
- Electrolab: <https://electrolab.com/>
- EKT in Beirut : <http://www.katranji.com/>

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

Board used in this course: Arduino UNO

It uses ATmega328P microcontroller

- Runs at 16 MHz
 - The CPU clock ticks 16 million times/second
- During one clock cycle the CPU executes one basic instruction
- Arduino UNO has 14 Digital I/O and 6 Analog Inputs



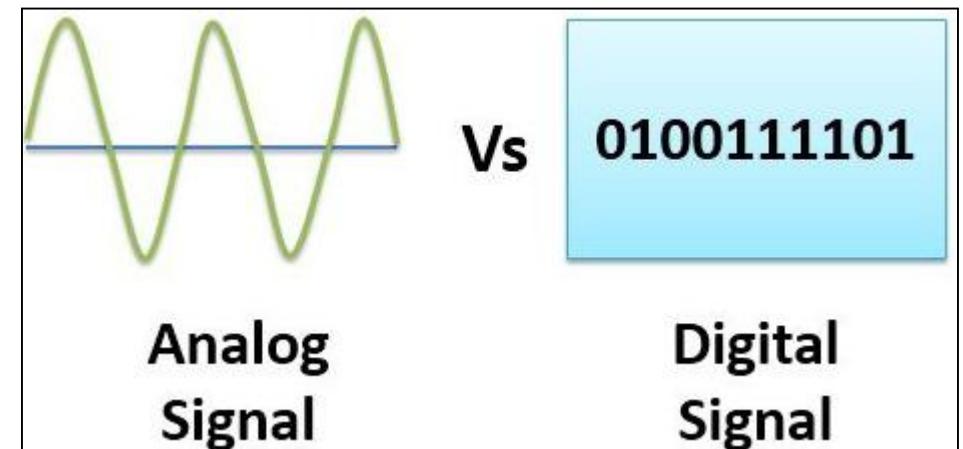
Microcontroller

- Hertz is a measure of frequency or how often something happens per second
- 1 KHz = 10^3Hz ; 1 MHz = 10^6Hz ; 1 GHz = 10^9Hz

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

ANALOG VS DIGITAL

- **An analog signal:** is any continuous signal for which the time varying feature.
 - For example, in an analog audio signal, the instantaneous voltage of the signal varies continuously with the pressure of the sound waves.
 - The term analog signal usually refers to electrical signals. However, mechanical, pneumatic, hydraulic, human speech, and other systems may also convey or be considered analog signals.
- **Digital signal:** A digital signal is a signal that represents data as a sequence of discrete values zeros or ones. Language of electronic platforms.



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

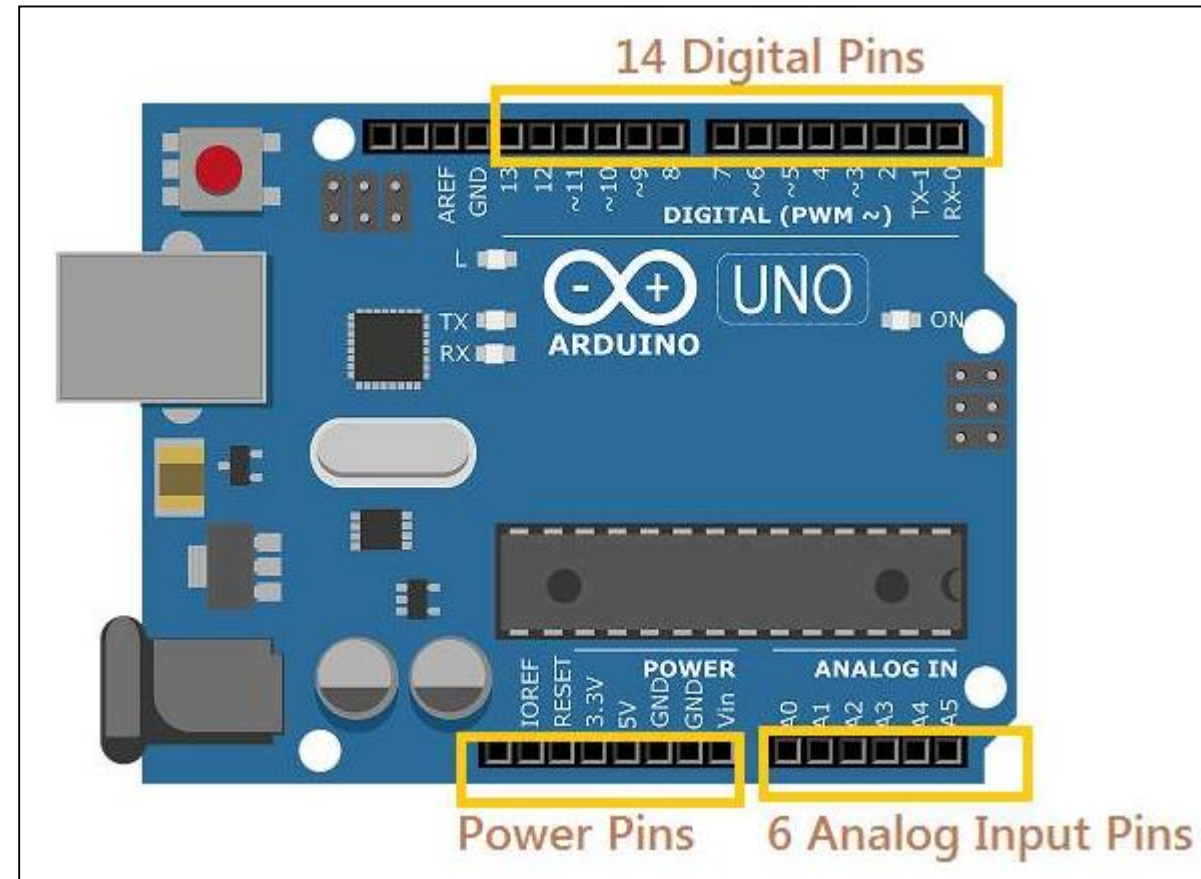
DIGITAL/ANALOG PINS

❖ Digital pins: You use digital pins to send and receive **digital signals**. Digital implies that the pins have **two states**: OFF and ON

- **OFF → 0 Volts**

- **ON → 5 Volts** *No values in between.*

❖ Analog IN pins: You use analog in pins to receive an **analog value**. An analog value is taken from a **range of values**. In this case, **the range is the same 0V to 5V as with the digital pins, but the value can be at any point**: 0.1, 0.2, 0.3, and so on



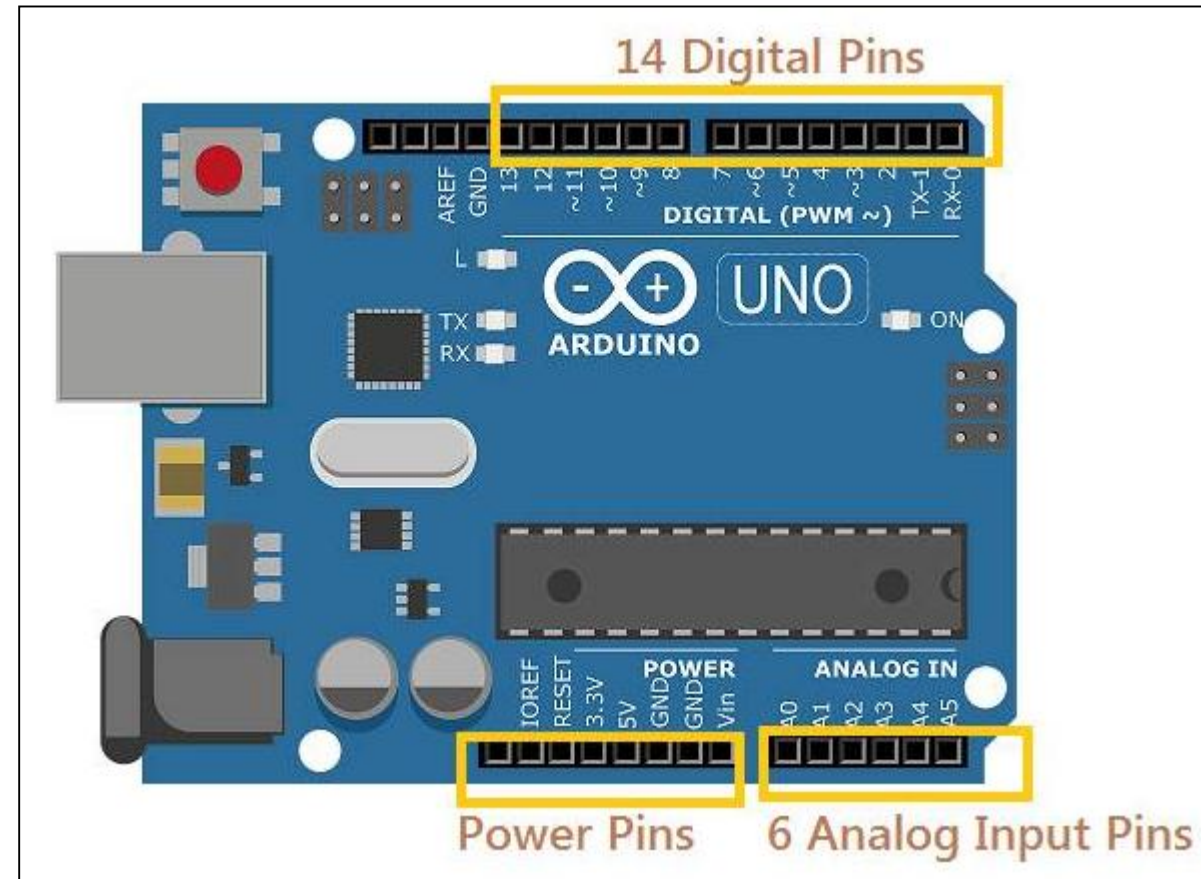
The analog pins, A0 to A5, measure voltages between 0V and 5V and convert analog signals to digital values (ADC).

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

DIGITAL/ANALOG PINS

❖ Analog OUT pins:

- The shrewd ones among you may have noticed that there seem to be no analog out pins.
- In fact, the analog out pins are there, but they're hidden among the digital pins marked as PWM using the ~ symbol.
- PWM stands for Pulse-Width Modulation, which is a technique you can use to give the impression of an analog output using digital pins.
- The ~ symbol appears next to digital pins 3, 5, 6, 9, 10, and 11, showing that you have six pins that are capable of PWM.

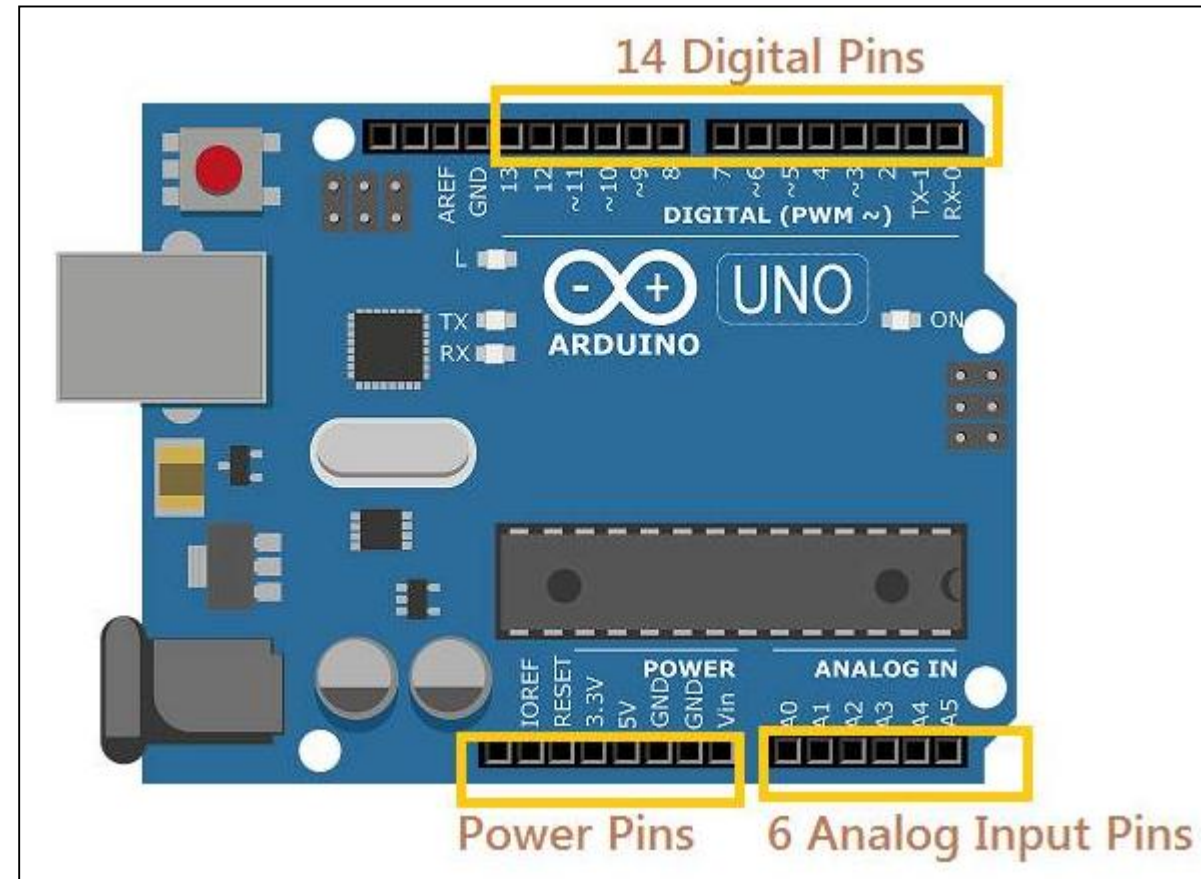


ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

DIGITAL/ANALOG PINS

❖ Power pins:

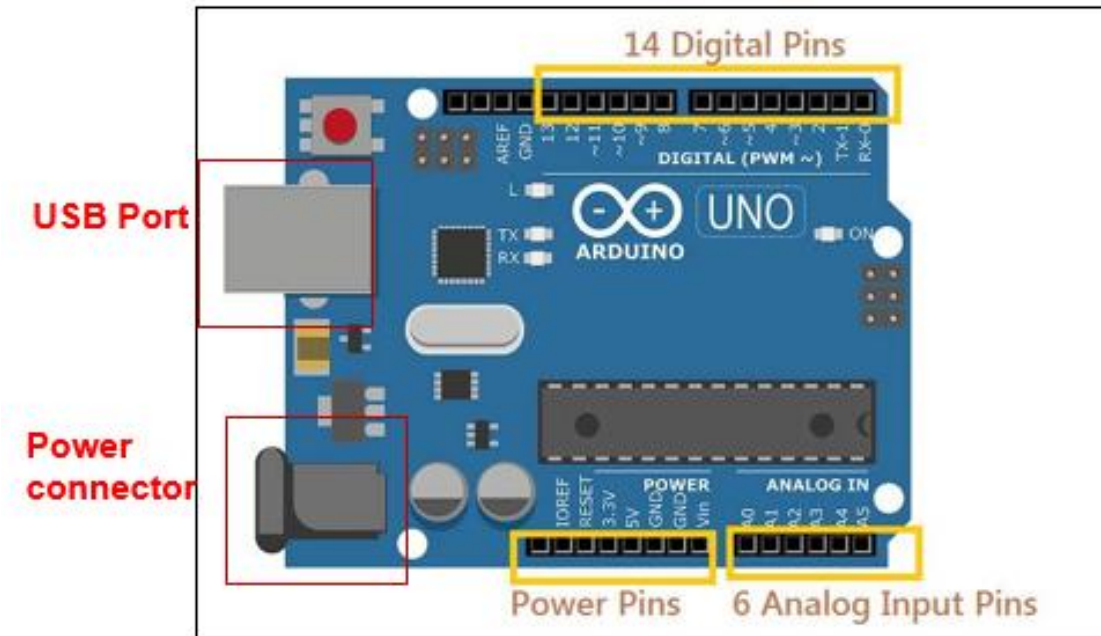
- You use the power pins to distribute power to inputs and outputs wherever power is needed.
- Vin, which stands for Voltage In, can be used to supply a voltage (V) equal to the one supplied by the external supply jack (for example, 12V). You can also use this pin to supply power to the Arduino from another source.
- GND marks the ground pins, which are essential to complete circuits. There is also a third ground by pin 13. All these pins are linked and share the same (called common) ground
- You can use 5V or 3.3V to supply a 5-volt or 3.3-volt power supply, respectively, to components or circuits.



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

USB & POWER SOCKETS

- ❖ To tell the microcontroller on the Arduino board what to do, you need to send a program to it. On the Uno, you send programs primarily by a USB connection.
- ❖ The Arduino uses the USB both for power and to transfer data. Using a USB cable is perfect for low-power applications and when data is being sent to or received from a computer.
- ❖ Next to the USB socket is another socket; this one is for power. This socket allows you to power your Arduino from an external power supply.
- ❖ The supply could be from an AC-to-DC adaptor (similar to those used on other consumer electronics), a battery, or even a solar panel.



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

LEDS

- ❖ The Uno board has four LEDs (Light-Emitting Diodes) labeled L, RX, TX, and ON. A LED is a component that produces light when electrical current flows through it.

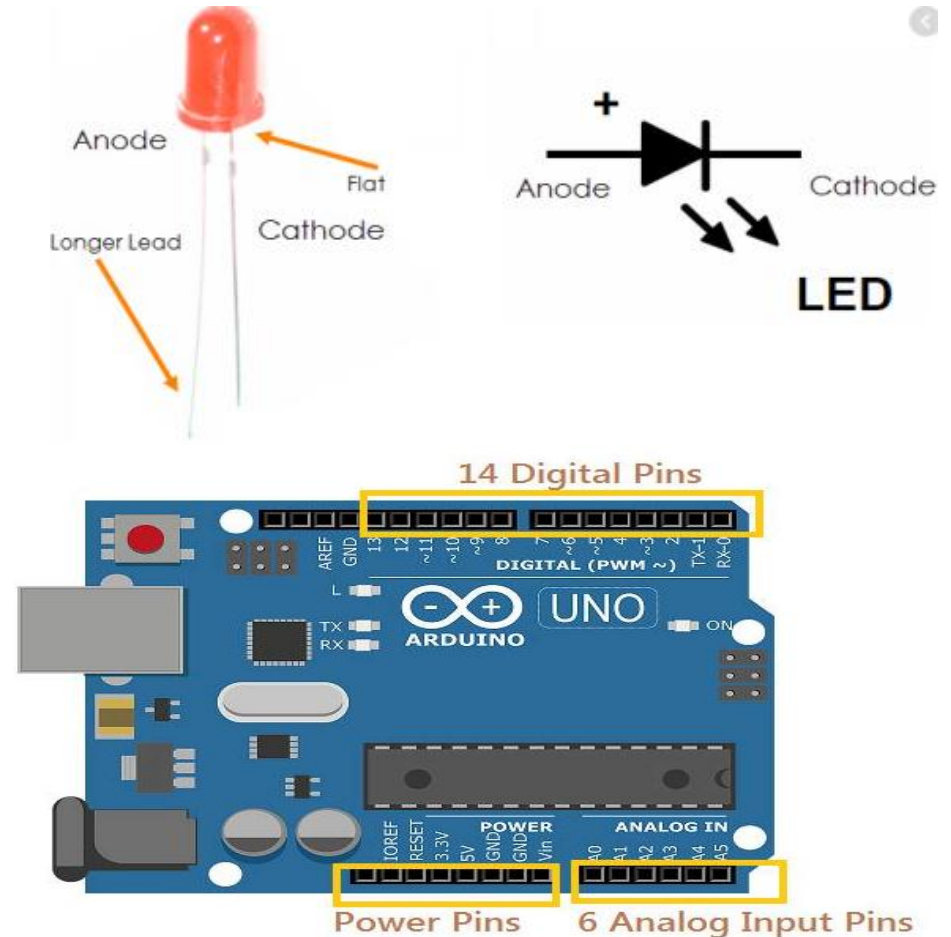
These four LEDs are used to indicate activity on the board, as follows:

- » ON is green and signifies that your Arduino is powered.
- » RX and TX tell you that data is being received or transmitted by the board.
- » L is a special LED connected to digital pin 13. It is great for testing to see whether your board is functioning as you want.

If your Arduino is plugged in but you don't see any lights, double-check that:

- » Your USB cable is plugged in
- » Your USB port is working — try another device in the port
- » Your cable is working — try another cable, if possible

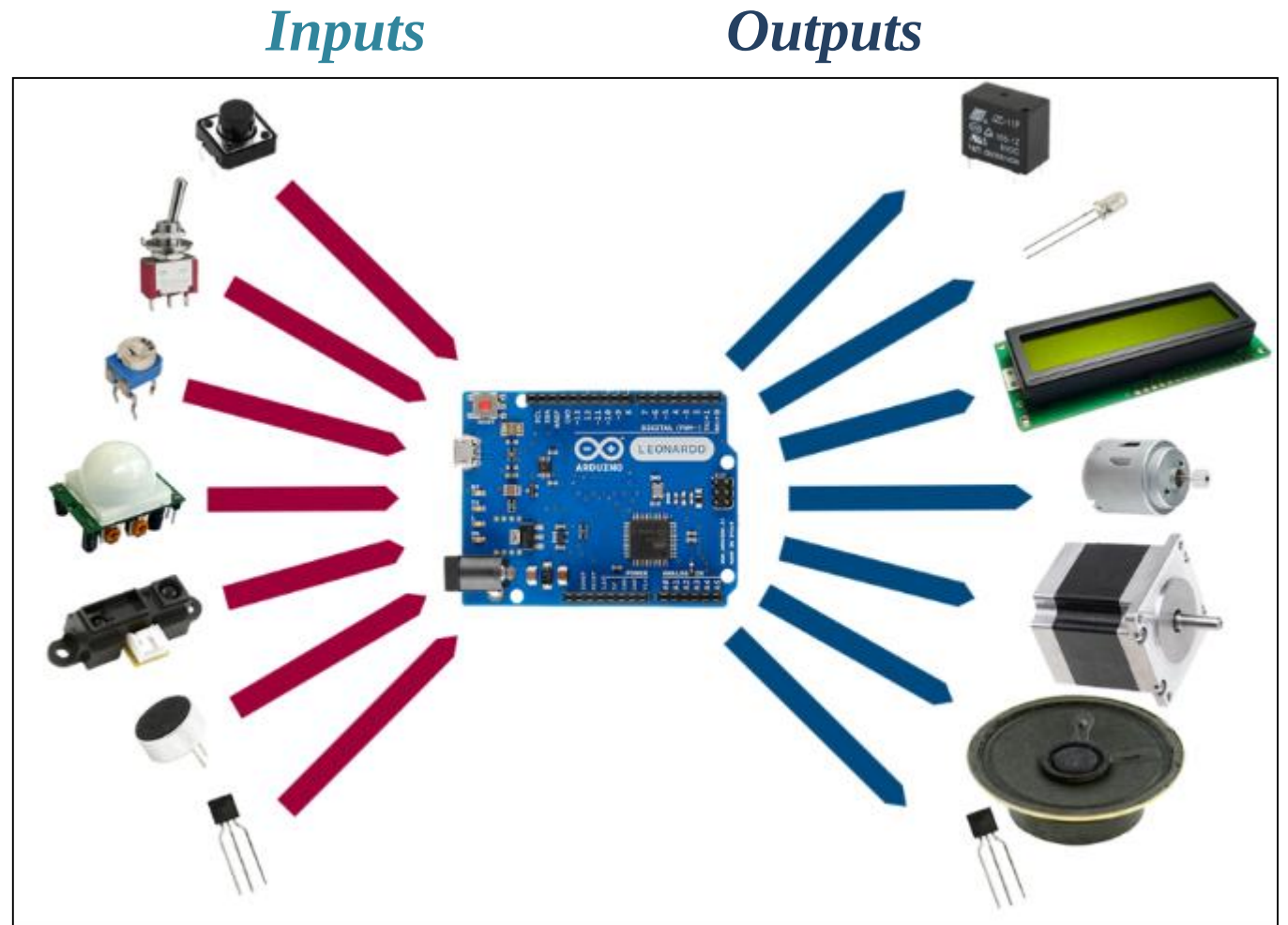
If the LED still doesn't illuminate, something is probably wrong with your Arduino. Your first destination should be the Arduino troubleshooting page at <http://arduino.cc/en/Guide/troubleshooting>. If you still have no luck, request a replacement Arduino from where you purchased the device.



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

INPUTS/ OUTPUTS

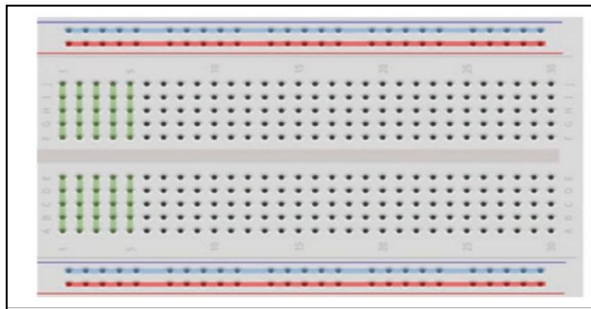
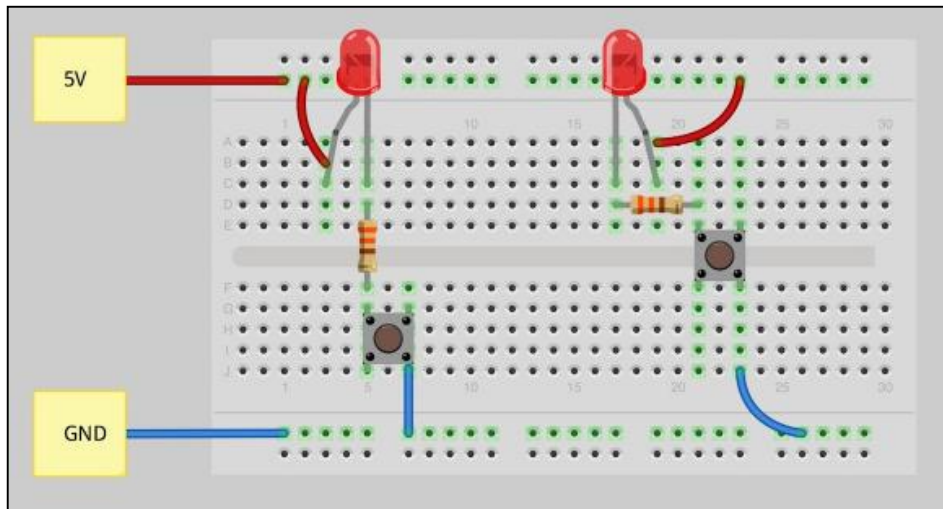
- Inputs are senses for your Arduino. At its most basic, an input could be a switch, such as a light switch in your home. It could be also a gyroscope, telling the Arduino the exact direction it's facing in three dimensions.
- An output could be the vibration of a cellphone, or it could be a huge visual display on the side of a building that can be seen for miles around.



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

BREADBOARDS

The solderless breadboard contains columns of connected sockets for positioning electronic components to create a circuit and for connecting to the Arduino



The two rows of the breadboard are used to connect to power (red) or ground (blue) lines in a circuit.

Holes in each short column (green) of the breadboard are connected together, but the columns are not connected so that two components each with one “leg” in the same green column are connected together.

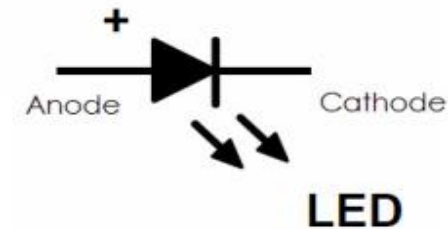
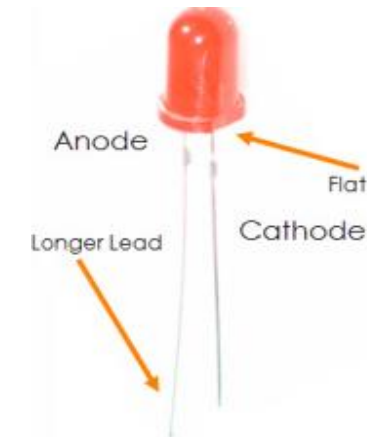
The middle area in the breadboard separates the breadboard into two unconnected halves.

Breadboards come in a variety of sizes.

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

LEDS

- Light Emitting Diodes (LEDs) - A type of diode allows current to pass in only **one direction**.
 - longer leg is connected to power (+)
 - the shorter leg to GND
- The LED should be placed in series with a resistor (R) to avoid its damage .
- The resistor should be at least 150 ohm; R = 220 ohm is used.
- Please refer to your book to know how to get the ohm value of a resistor.
- LED's maximum permitted current of $I_{max} = 20\text{mA}$.
- $U_{min}^{LED} = 2\text{ V}$
- Since Arduino supplies $5\text{ V} > 2\text{ V}$, the LED should be placed in series with a resistor (R) to avoid its damage .
- $U_R = 5 - 2 = 3\text{ V}$
- $R = \frac{3}{0.02} = 150\text{ ohm}$
- The resistor should be at least 150 ohm; R = 220 ohm is used.



ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

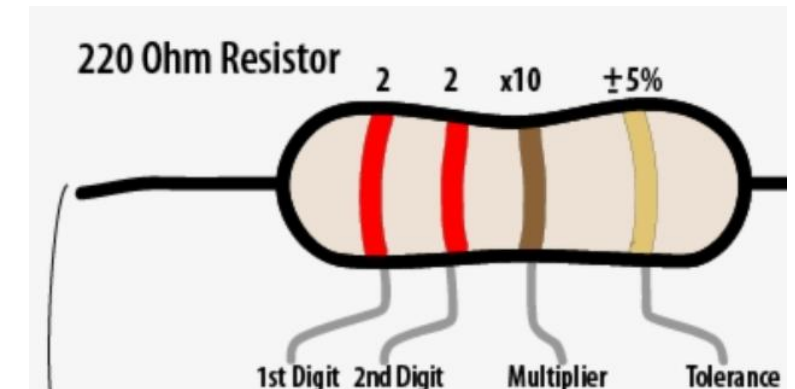
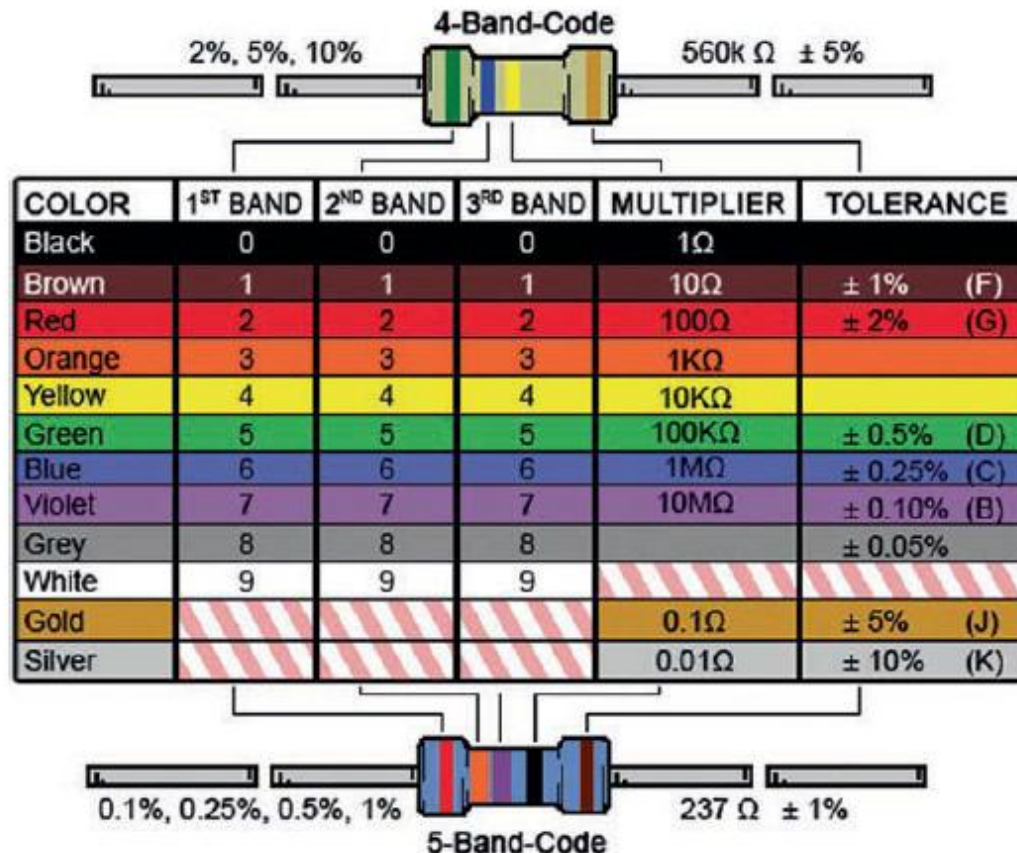
LEDS

- The Arduino supplies a regulated 5V output from the pin marked 5V, but a resistor is required to ensure that the current does not exceed the LED's maximum permitted current of 20mA. Without the resistor, the high current would damage the LED.
- The forward voltage drop across the LED is 2V, which is the minimum voltage required to turn on the LED. With a 5V output from the Arduino, there is $3V = 5V - 2V$ across the resistor. If the current through the resistor and the LED is to be at most 20mA, then from Ohm's law, the resistor value $(R = V/I) = 3/0.02 = 150\Omega$.
- A resistor of at least 150Ω would protect the LED from an excessively high current and the widely available 220Ω resistor can be used.

ARDUINO BOARDS: CATEGORIES, SPECIFICATIONS, SCHEMATICS

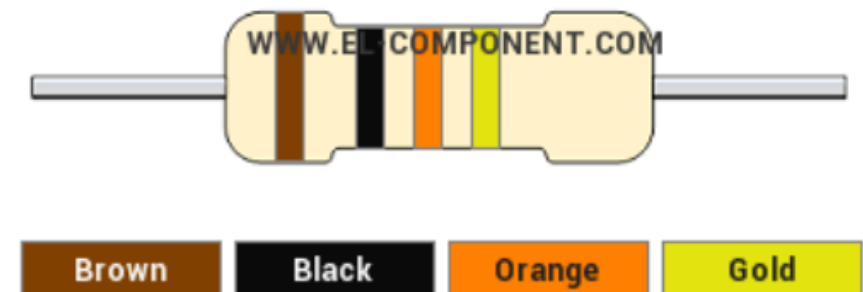
RESISTOR COLOR CODES

- Resistors are color-coded to identify the resistance.



The color code is red – red – brown – gold.

10k Ohm Resistor, 5% tolerance



REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

DATA TYPES

- Data types in C programming language, which Arduino uses refer to an extensive system used for declaring variables or functions of different types.
- The type of a variable determines how much space it occupies in the storage and how the bit pattern stored is interpreted.
- **Void** : The void keyword is used only in function declarations. It indicates that the function is expected to return no information to the function from which it was called.

```
Void Loop ( ) {  
    // rest of the code  
}
```

- **Boolean** : holds one of two values, true or false. Each Boolean variable occupies one byte of memory. 1 Byte = 8 bits

```
boolean val = false ; // declaration of variable with type boolean  
boolean state = true ; // declaration of variable with type boolean
```

REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

DATA TYPES

- **Char** : A data type that takes up one byte of memory that stores a character value. Character literals are written in single quotes like this: 'A' and for multiple characters, strings use double quotes: "ABC".

```
Char chr_a = 'a' ;//declaration of variable with type char and initialize
```

- **Byte** : stores an 8-bit unsigned number, from 0 to 255.

```
byte m = 25 ;//declaration of variable with type byte
```

- **Int** : Integers are the primary data-type for number storage. int stores a 16-bit (2-byte) value. This yields a range of -32,768 to 32,767 (minimum value of -2^{15} and a maximum value of $(2^{15}) - 1$).

```
int counter = 32 ;// declaration of variable with type int and initialize it
```

REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

DATA TYPES

- **Unsigned int** : Unsigned ints (unsigned integers) are the same as int in the way that they store a 2 byte value. Instead of storing negative numbers, however, they only store positive values, yielding a useful range of 0 to 65,535 ($2^{16} - 1$).

```
Unsigned int counter = 60 ; // declaration of variable with  
type unsigned int and initialize it with 60
```

- **Float** : Data type for floating-point number is a number that has a decimal point. Floating-point numbers are often used to approximate the analog and continuous values because they have greater resolution than integers. Floating-point numbers can be as large as 3.4028235E+38 and as low as -3.4028235E+38. They are stored as 32 bits (4 bytes) of information.

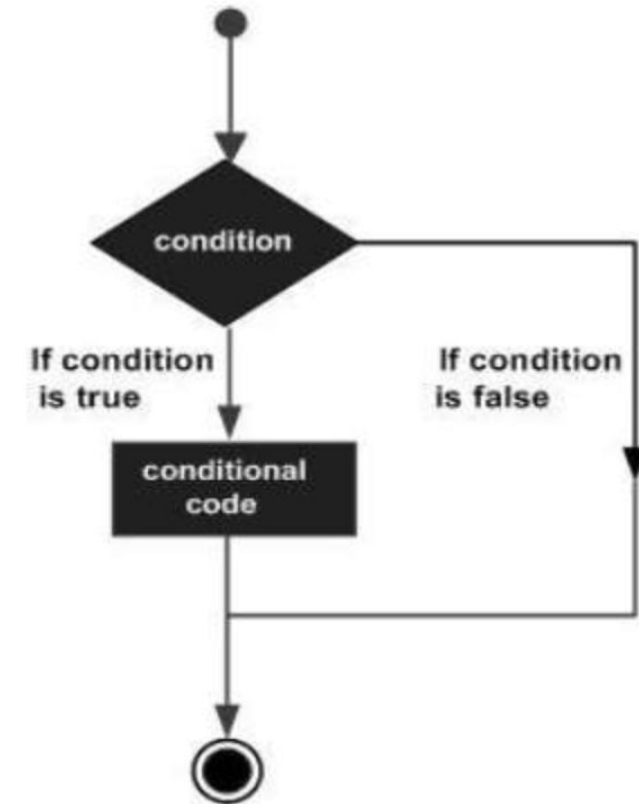
```
float num = 1.352; // declaration of variable with type float
```

REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

CONTROL STATEMENTS

- Control Statements are elements in Source Code that control the flow of program execution.
- Decision making structures require that the programmer specify one or more conditions to be evaluated or tested by the program.
- It should be along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.
- Following is the general form of a typical decision making structure found in most of the programming languages

```
if (condition) //condition: a boolean expression (i.e., can be true or false).  
{  
    //statement(s) executed if the condition is true  
}  
else  
{  
    //statement(s) executed if the condition is false  
}
```



REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

CONTROL STATEMENTS-IF STATEMENT VARIATIONS

- **if statement looks like**

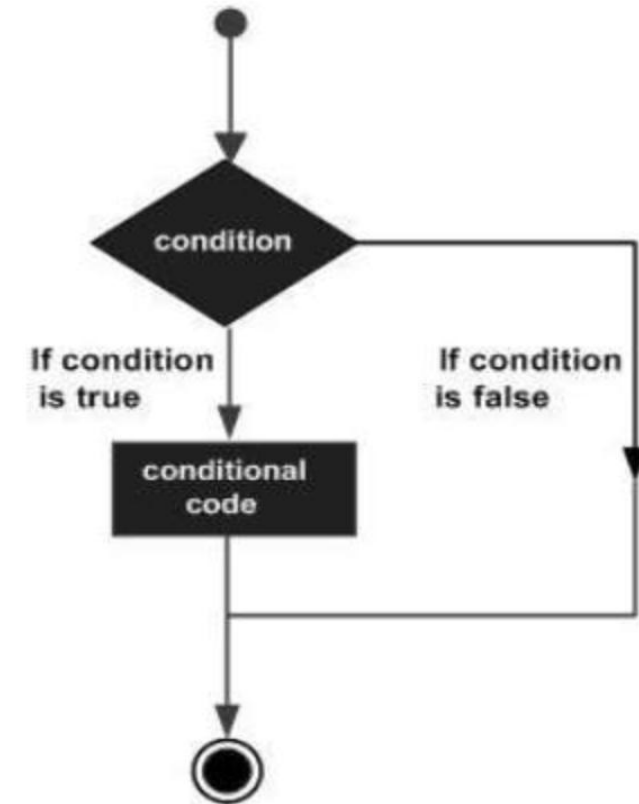
```
if (someCondition) {  
    // do instructions if the condition is true  
}
```

- **There is a common variation called if-else that looks like this:**

```
if (someCondition) {  
    // do instructions if the condition is true  
} else {  
    // do instructions if the condition is false  
}
```

- **There's also the else-if, where you can check a second condition if the first is false:**

```
if (someCondition) {  
    // do instructions if the condition is true  
} else if (anotherCondition) {  
    // do instructions only if the first condition is false and the second condition is true  
}
```



REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

OPERATORS

- An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators.

We will focus on the comparison and Boolean operators.

- **Boolean operators** give you some control over the flow of your program.

Operator	Description
!	Logical NOT results in a true if the operand is false and vice versa.
&&	Logical AND results in true only if both operands are true.
	Logical OR results in a true if either of the two operands is true.

REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

OPERATORS

- Comparison operators**

Operator name	Operator simple	Description	Example
equal to	==	Checks if the value of two operands is equal or not, if yes then condition becomes true.	(A == B) is not true
not equal to	!=	Checks if the value of two operands is equal or not, if values are not equal then condition becomes true.	(A != B) is true
less than	<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true
greater than	>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true
less than or equal to	<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true
greater than or equal to	>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true

REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

OPERATORS

- **Example**

```
if (digitalRead(2) == HIGH && digitalRead(3) == HIGH)
{ // if BOTH the switches read HIGH
  // execute these instructions
}
```

```
if (x > 0 || y > 0)
{ // if either x or y is greater than zero
  // execute these instructions
}
```

```
if (!x)
{ // if x is not true
  // execute these instructions
}
```

REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

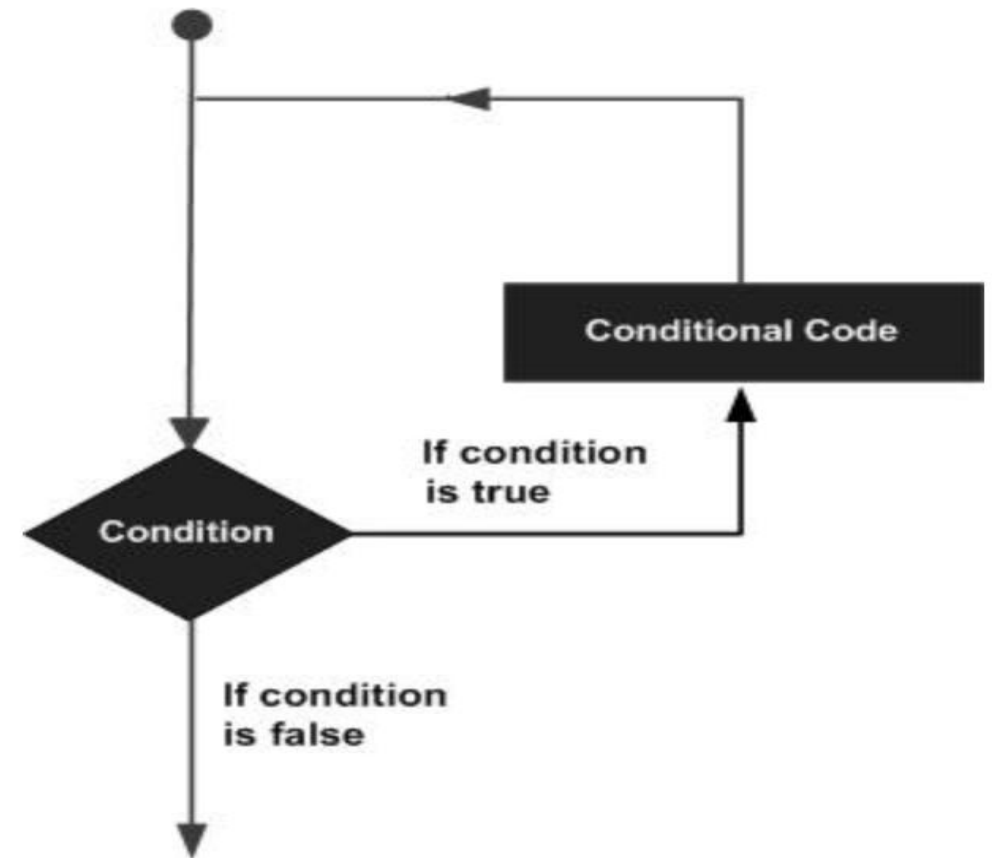
LOOP

A loop statement allows us to execute a statement or group of statements multiple times. Following is the general form of a loop statement in most programming languages.

The for loop is used to repeat a block of statements enclosed in curly braces a predefined number of times.

```
for (initialization; condition; increment) {  
    // instruction(s);  
}
```

- **initialization:** happens first and exactly once.
- **condition:** each time through the loop, the condition is tested; if it's true, the instructions block is executed. When the condition becomes false, the loop ends.
- **increment:** executed each time through the loop when condition is true.



REVIEW OF C PROGRAMMING: SYNTAX, VARIABLES, OPERATORS, CONTROL STATEMENTS

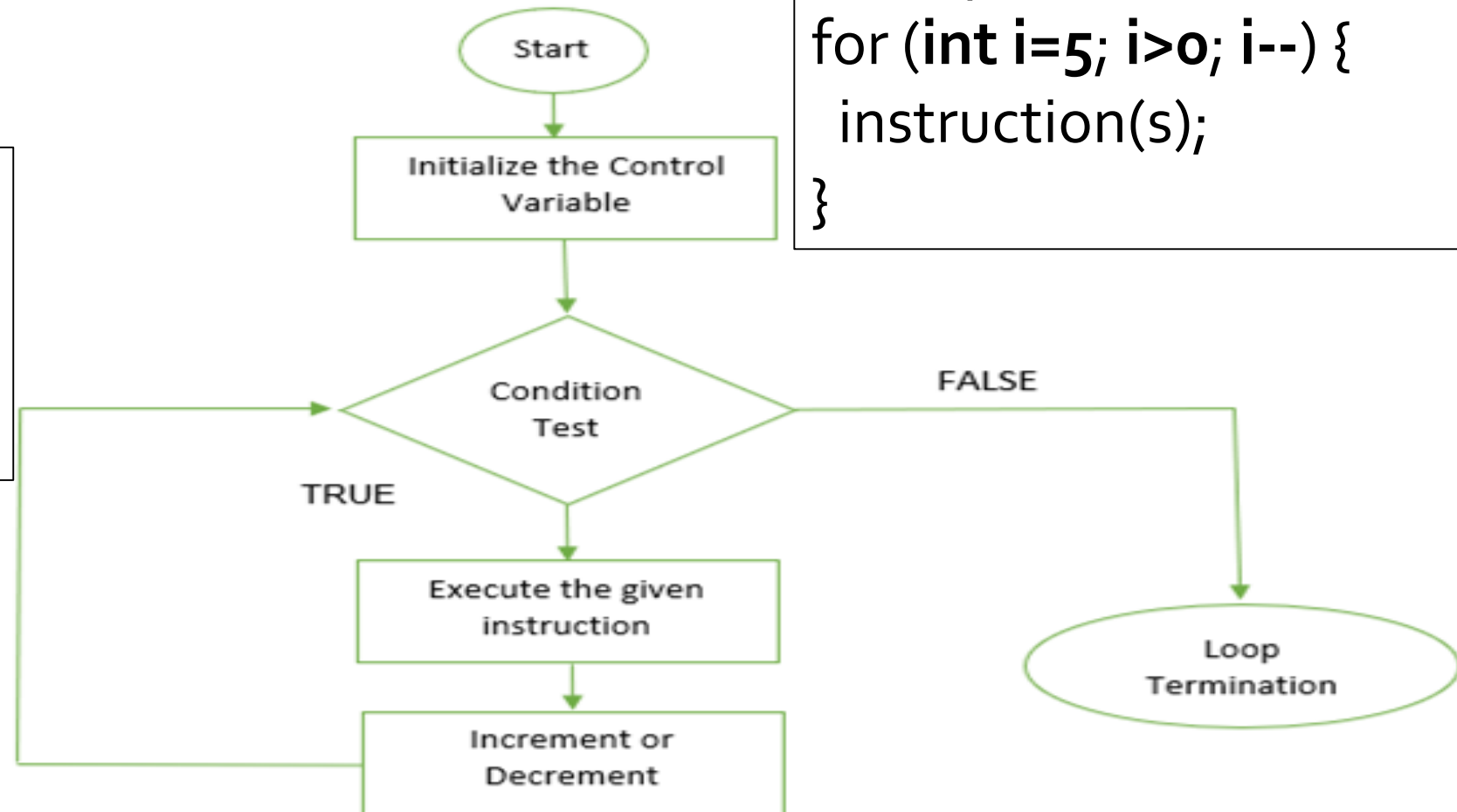
LOOP

Example 1:

```
for (int i=0; i<=9; i++) {  
    instruction(s);  
}
```

Example 2:

```
for (int i=5; i>0; i--) {  
    instruction(s);  
}
```



MODULE 2 : ARDUINO ECOSYSTEM & PROGRAMING BASICS

- Arduino boards: categories, specifications, schematics.
- Review of C programming: syntax, variables, operators, control statements.
- **Embedded toolchain: Arduino IDE, simulator setup.**
- Writing basic Arduino programs and testing.

EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

The Arduino software is a type of an Integrated Development Environment (IDE), a tool common in software development that allows you to write, test, and upload programs.

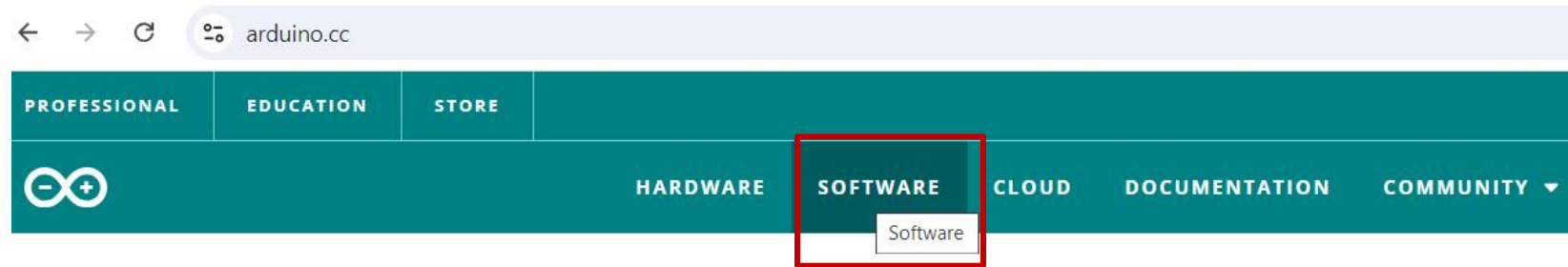
Versions of Arduino software are available for Windows, macOS, Linux, and Arduino Web Editor.

Arduino IDE features:

- Open-source Software
- Easy to write code
- Easy to upload the code to the board
- It runs on Windows, Mac OS, and Linux.
- The environment is written in Java, C++..
- It can be used with any Arduino board

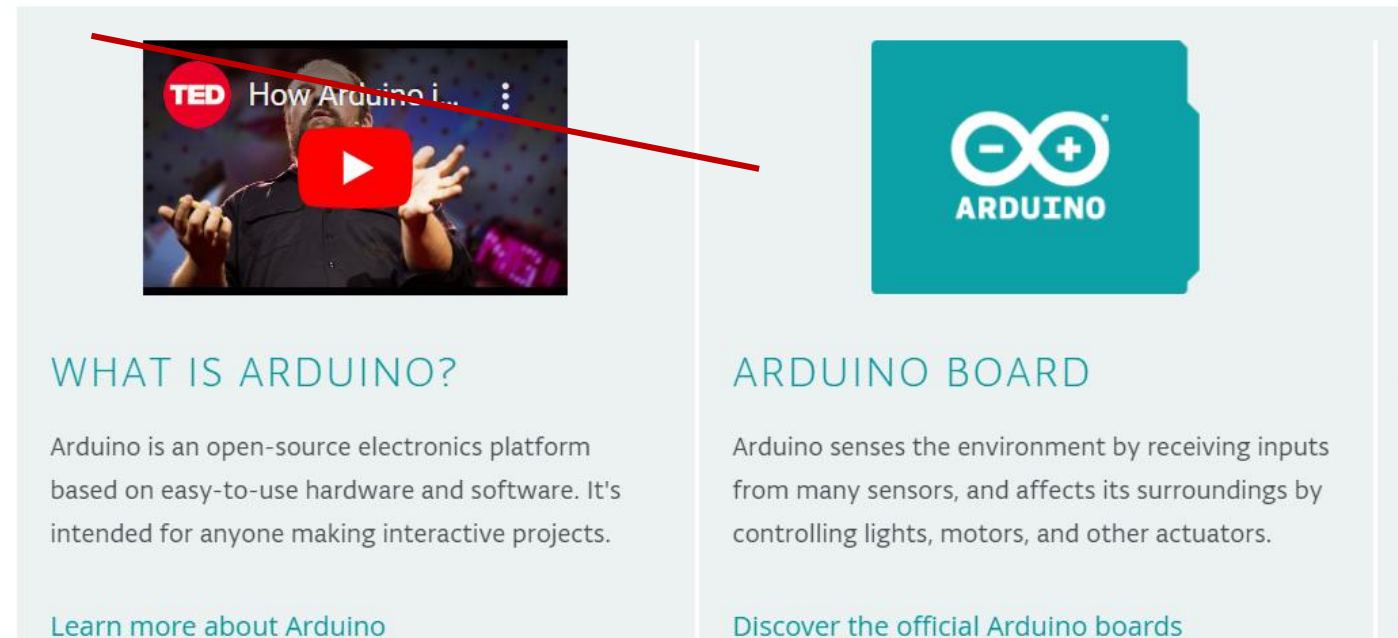
EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE



Step 1: Go to: <https://www.arduino.cc/>

Step 2: Click on Software



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

Step 3: Select the suitable Arduino version based on the operating system of your computer

[HARDWARE](#) [SOFTWARE](#) [CLOUD](#) [DOCUMENTATION](#) [COMMUNITY](#) [BLOG](#) [ABOUT](#)

Downloads



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

[SOURCE CODE](#)

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

Step 4: Download Arduino IDE

Download Arduino IDE & support its progress

Since the 1.x release in March 2015, the Arduino IDE has been downloaded **86,509,426** times — impressive! Help its development with a donation.

\$3	\$5	\$10	\$25	\$50	Other
-----	-----	------	------	------	-------

CONTRIBUTE AND DOWNLOAD

or

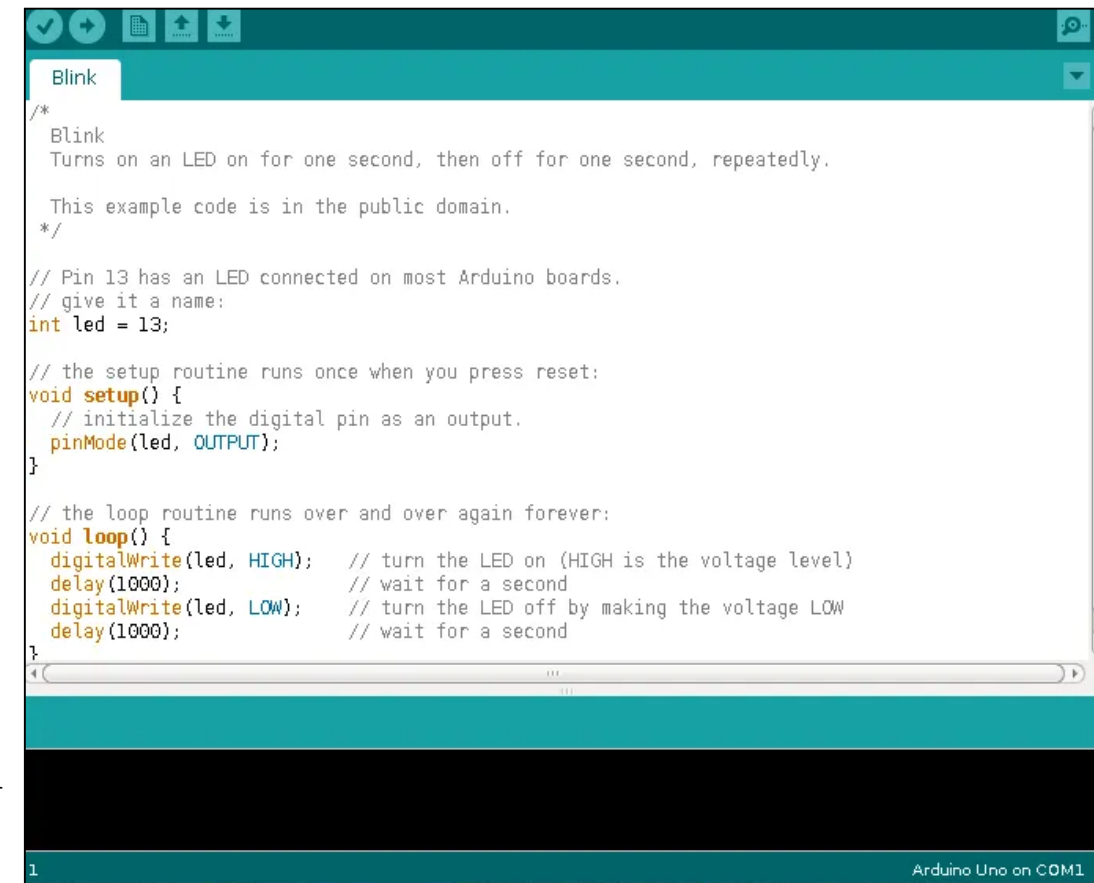
JUST DOWNLOAD



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

- Programs written for Arduino are known as sketches.
- The Arduino software is presented to you as a graphical user interface, or GUI (pronounced “goo-ey”).
- A GUI provides a visual way of interacting with a computer.
- Without it, you would need to read and write lines of text, like the DOS prompt in Windows, Terminal in macOS.

A screenshot of the Arduino IDE interface. The top bar is teal with icons for checkmark, undo, redo, new file, open file, and save file. Below the bar, a tab labeled 'Blink' is active. The main text area contains C++ code for a blink sketch. The code includes comments explaining the purpose and pin configuration, and defines the setup and loop functions. The status bar at the bottom shows '1' on the left and 'Arduino Uno on COM1' on the right.

```
/*
Blink
Turns on an LED on for one second, then off for one second, repeatedly.

This example code is in the public domain.
*/

// Pin 13 has an LED connected on most Arduino boards.
// give it a name:
int led = 13;

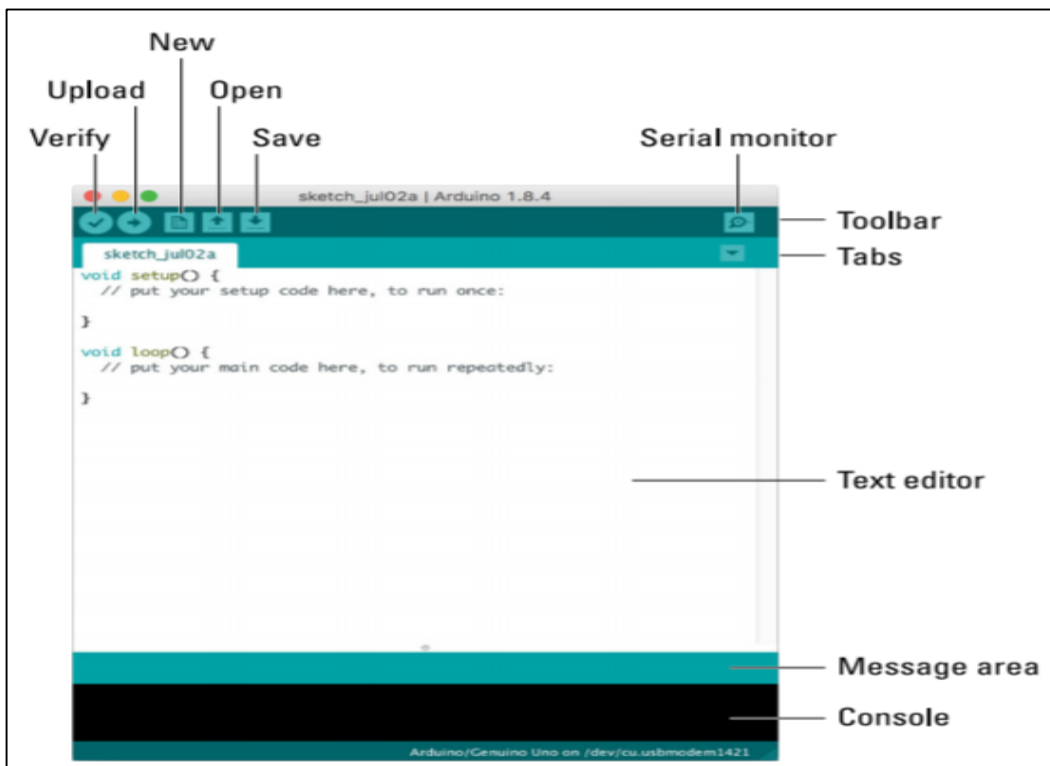
// the setup routine runs once when you press reset:
void setup() {
  // initialize the digital pin as an output.
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  digitalWrite(led, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);               // wait for a second
  digitalWrite(led, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);               // wait for a second
}
```

EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

The turquoise window is Arduino's GUI. It's divided into the following four main areas :



» **Toolbar:** The toolbar contains several buttons that are commonly needed when writing sketches for Arduino. These buttons, which are also available on the menu bar, perform the following functions:

- *Verify:* Checks that your code makes sense to the Arduino software. Known as *compiling*, this process is a bit like a spelling and grammar checker. Be aware, however, that although the compiler checks that your code has no obvious mistakes, it does not guarantee that your sketch works correctly.
- *Upload:* Sends your sketch to a connected Arduino board. It automatically compiles your sketch before uploading it.
- *New:* Creates a sketch.
- *Open:* Opens an existing sketch.
- *Save:* Saves the current sketch.
- *Serial monitor:* Allows you to view data that is being sent to or received by your Arduino board.

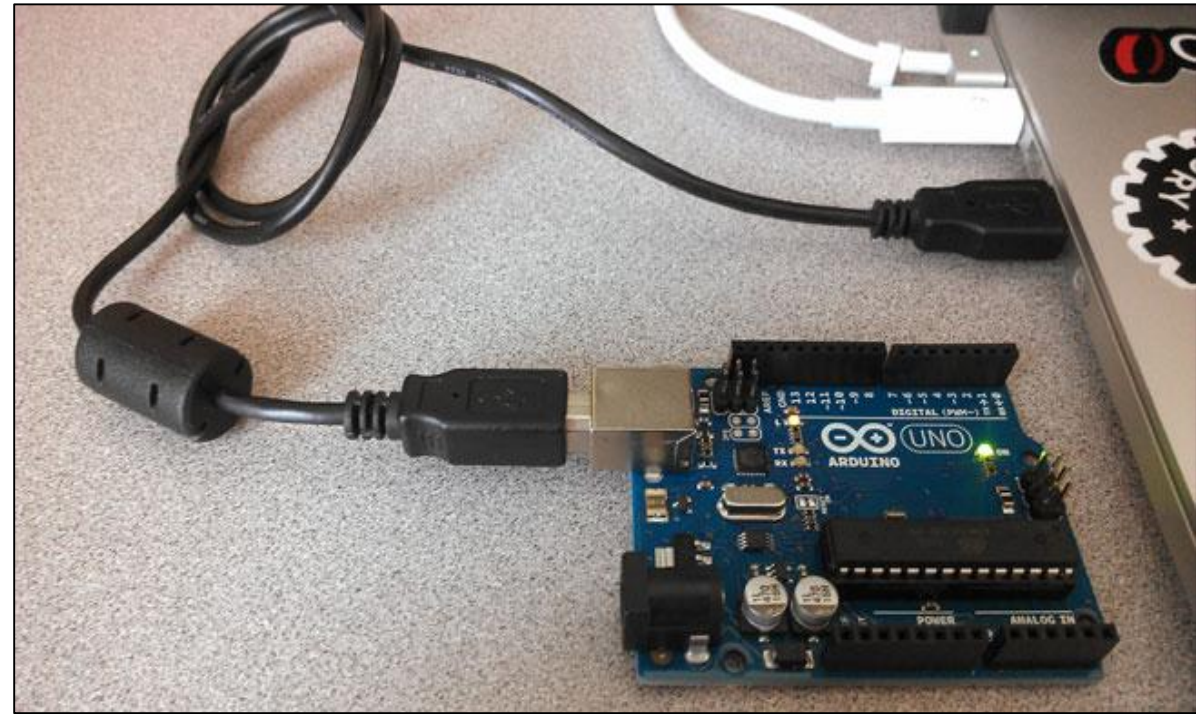
» **Text editor:** This area displays your sketch as text. It is almost identical to a regular text editor but has a few added features. Some text is color coded if the Arduino software recognizes it. You also have the option to auto-format the text so that it is easier to read.

» **Message area:** Even after years of using Arduino, you'll still make mistakes (everybody does), and the message area is the first way for you to find out that something is wrong. (**Note:** The second way is the smell of burning plastic.)

EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

- Connect the cable to the Arduino and the pc as shown in the picture
- Open Arduino IDE
- Select tools > ports > choose the port that the Arduino is connected to
- Select tools > board > Arduino Uno



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE - EXAMPLE: RUN THE BLINK SKETCH

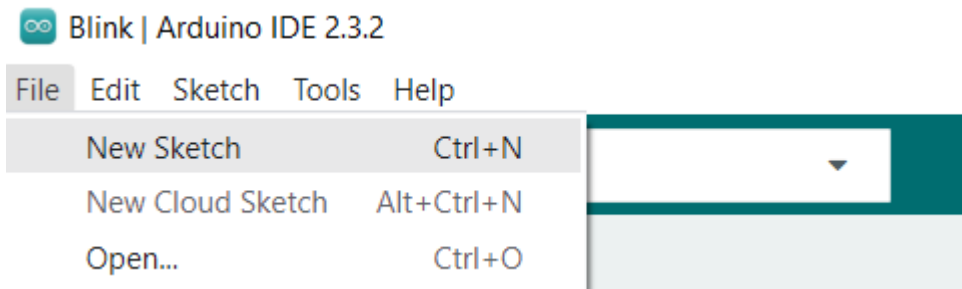
- Follow these steps to run the blink sketch.
 1. Connect the Arduino to a computer or laptop with the USB-to-serial cable.
 2. In Arduino IDE, select File -> Examples -> 01. Basics -> Blink.
 3. Click the Compile and Upload.
- The built-in LED on the Arduino will now flash every second.

- The **Problem uploading to board** error message indicates that the serial port should be updated. Select Tools -> Port and choose the appropriate port (for example, COM3 or COM4) for the Arduino. Go to step 3.
- The error message indicates **An error occurred while uploading the sketch** that the description of the microcontroller should be updated. Select Tools -> Board and choose the relevant board (for example, Arduino/ Genuino Uno). Go to step 3.

EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

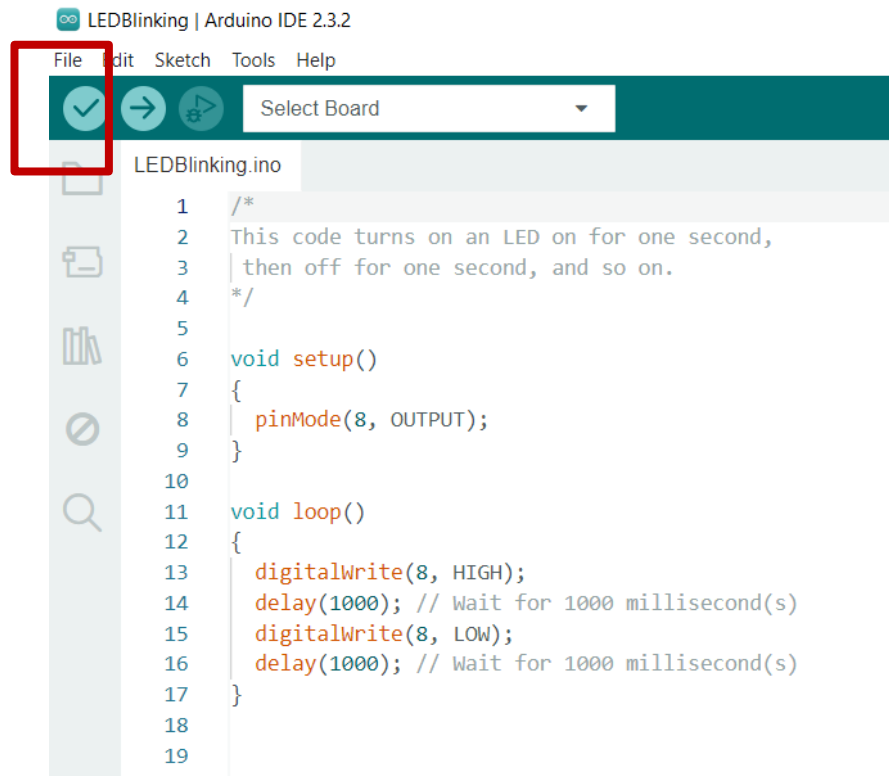
To write your code:



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

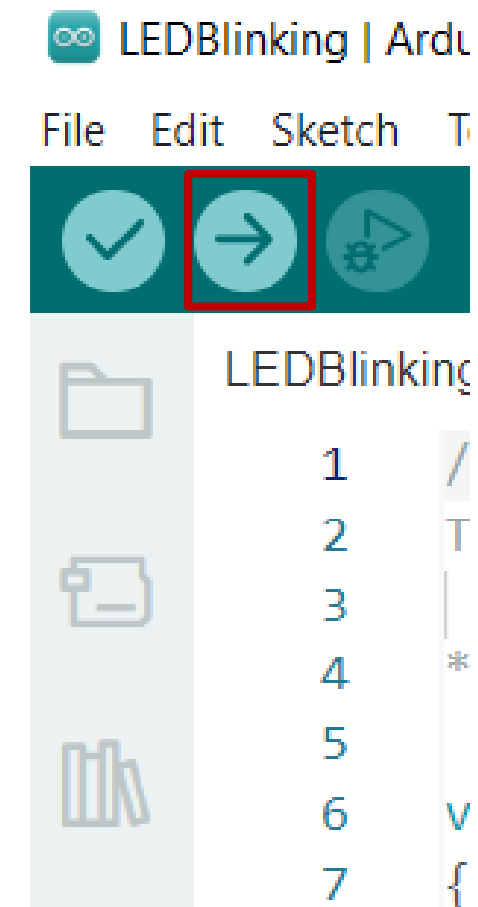
- Write your code and compile it to make sure it is correct.



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

ARDUINO IDE

1. Click on File → Save as to save and rename your file
2. Click on Compile to make sure that the code is correct
3. Connect the cable to the Arduino and the pc
4. Click Tools → Board → Arduino Uno
5. Then press on Port and choose ComX
6. After choosing the board and port, click on upload
7. Check the LED on the board, it will start blinking



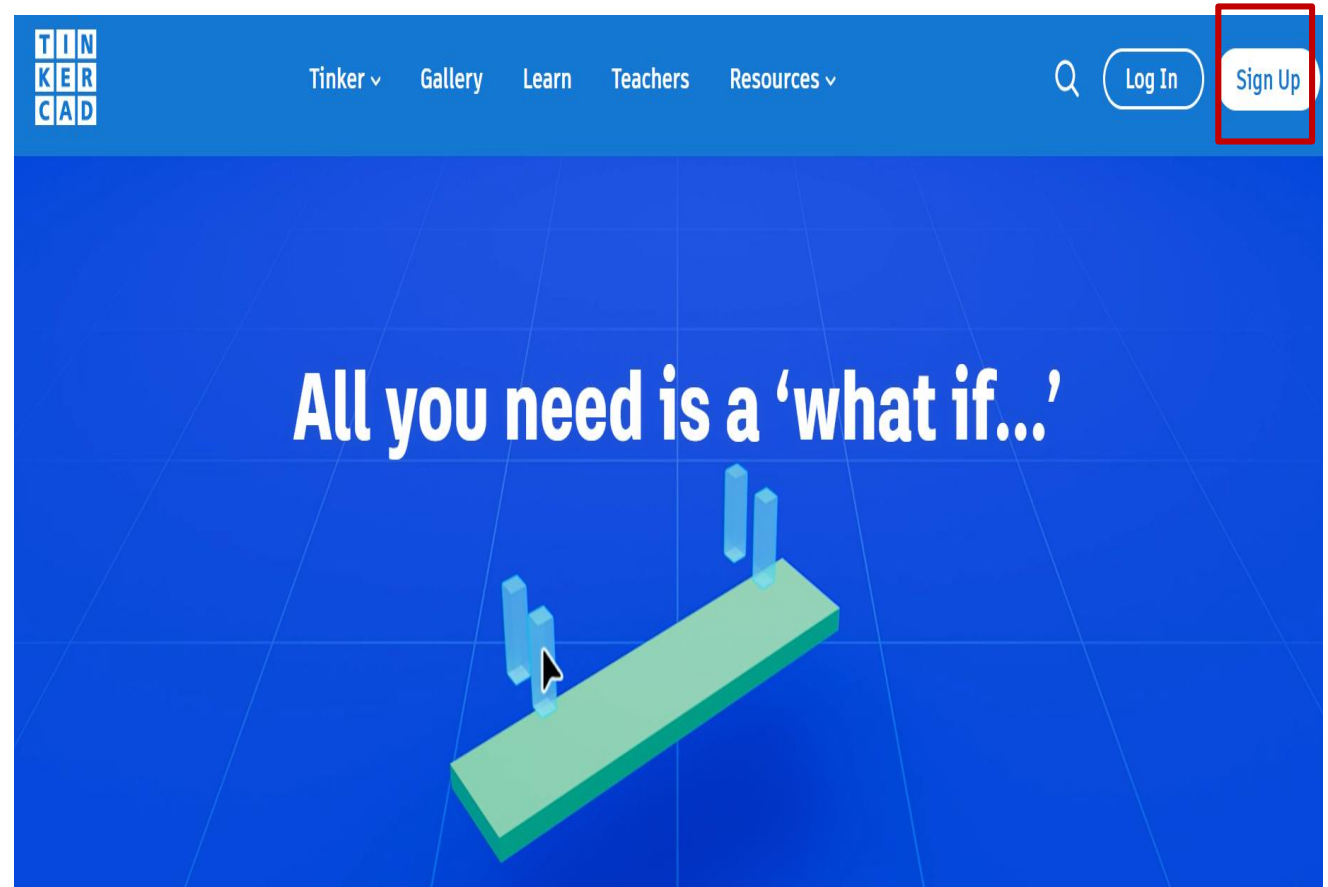
EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

SIMULATOR SETUP - TINKERCAD

Step 1: Go

to: <https://www.tinkercad.com/>

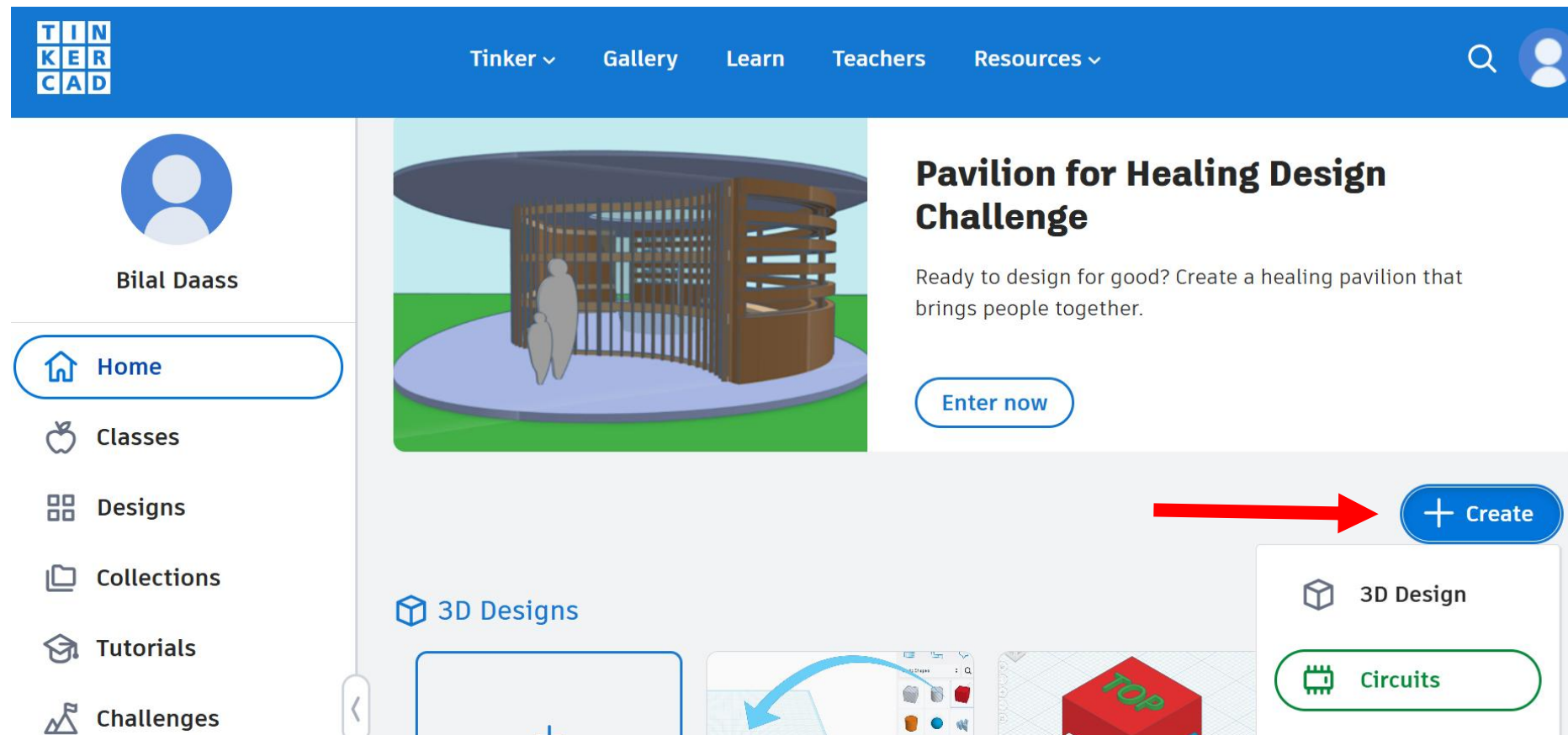
Step 2: Click on Login or Sign Up



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

SIMULATOR SETUP - TINKERCAD

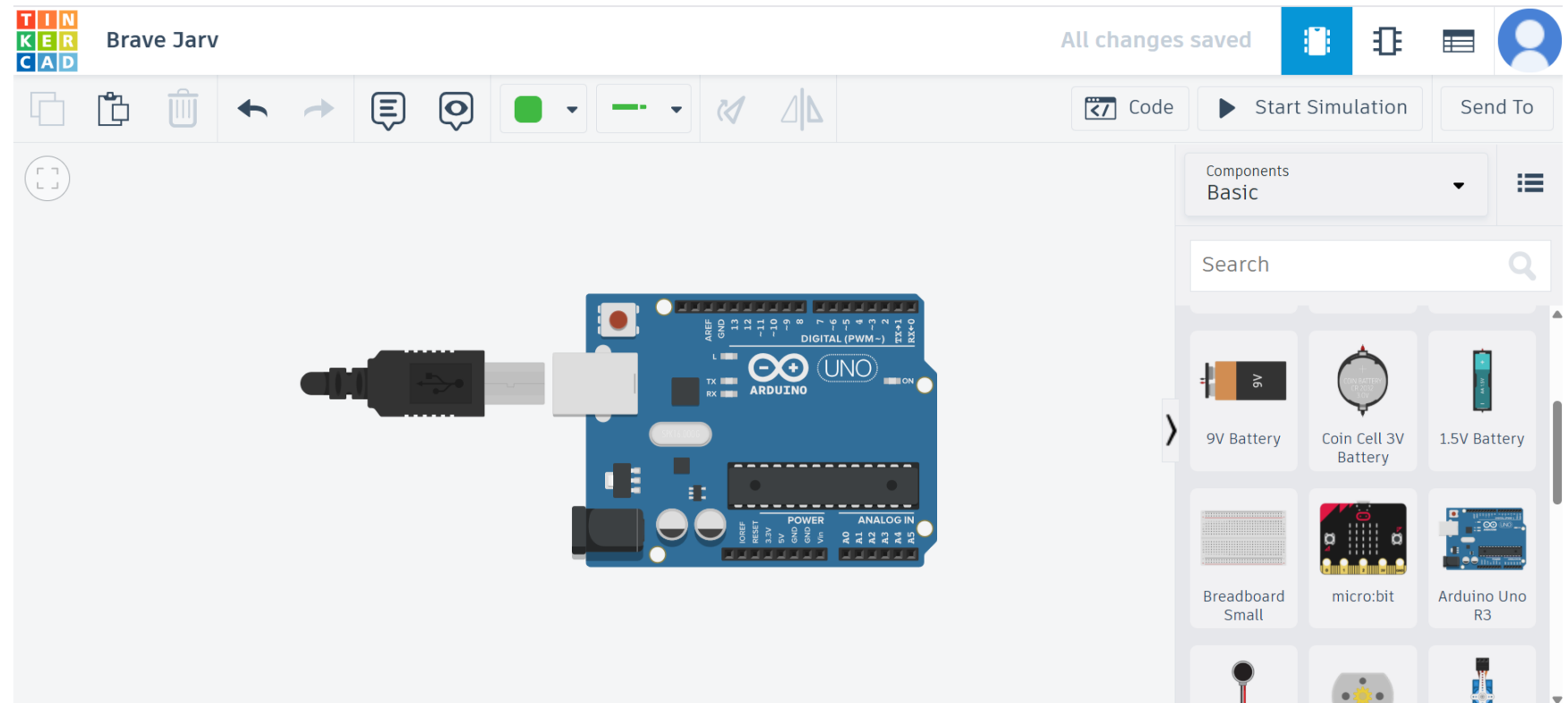
Step 3: Create new circuit



EMBEDDED TOOLCHAIN: ARDUINO IDE, SIMULATOR SETUP

SIMULATOR SETUP - TINKERCAD

Step 4 : Start simulating

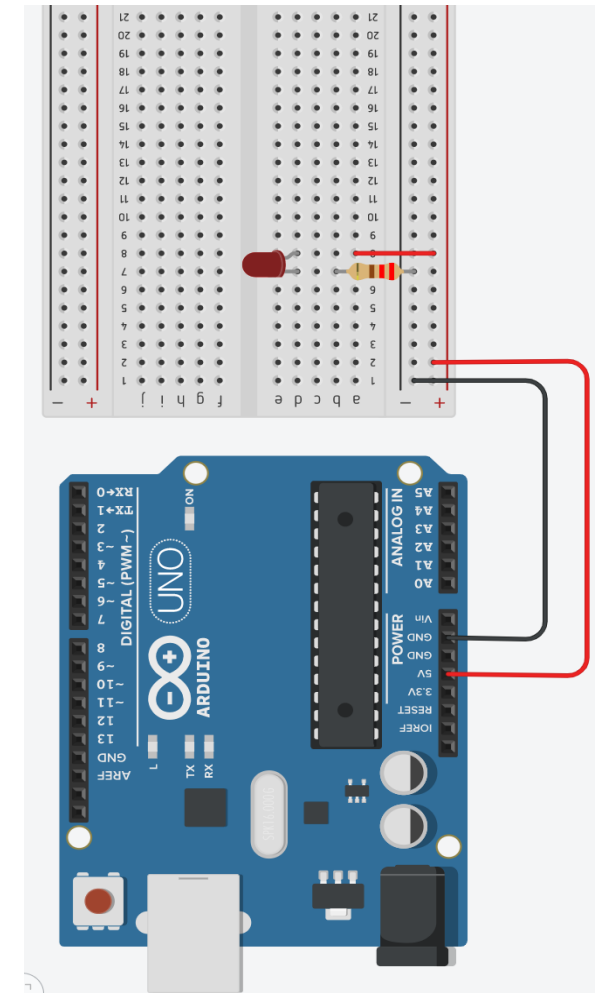
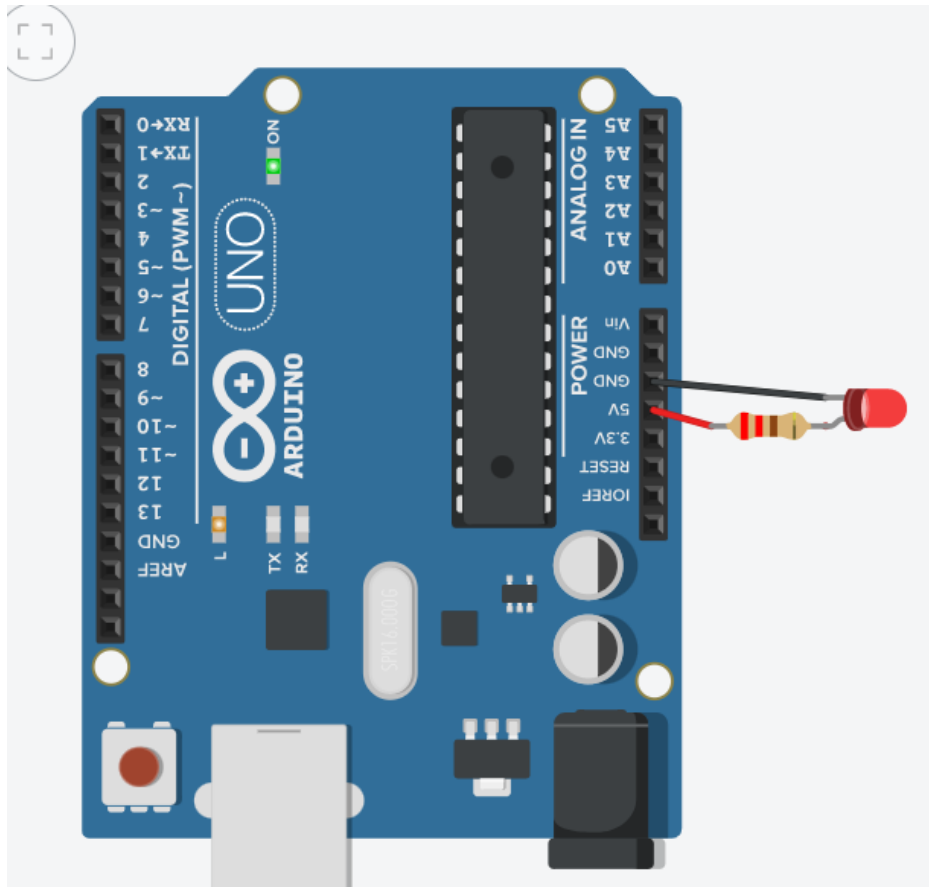


MODULE 2 : ARDUINO ECOSYSTEM & PROGRAMING BASICS

- Arduino boards: categories, specifications, schematics.
- Review of C programming: syntax, variables, operators, control statements.
- Embedded toolchain: Arduino IDE, simulator setup.
- **Writing basic Arduino programs and testing.**

WRITING BASIC ARDUINO PROGRAMS AND TESTING

Example 1: Simple LED circuits



WRITING BASIC ARDUINO PROGRAMS AND TESTING

ARDUINO SKETCH

An Arduino program is called a *sketch*. It's the unit of code that is uploaded to and run on an Arduino board.

/* Example of a sketch

This code turns on an LED on for one second,
then off for one second, and so on.

*/

```
void setup()
```

```
{
```

```
  pinMode(8, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
  digitalWrite(8, HIGH);
```

```
  delay(1000); // Wait for 1000 millisecond(s)
```

```
  digitalWrite(8, LOW);
```

```
  delay(1000); // Wait for 1000 millisecond(s)
```

```
}
```

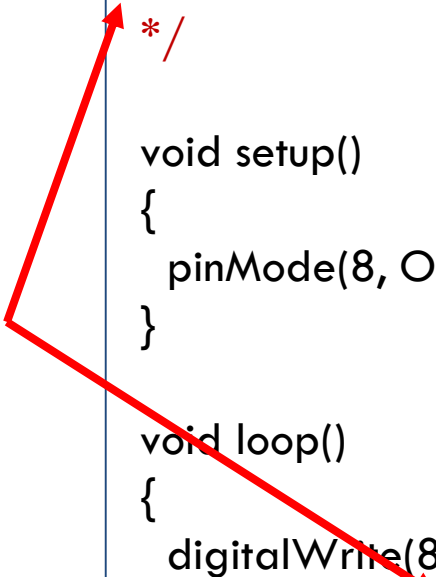
WRITING BASIC ARDUINO PROGRAMS AND TESTING

ARDUINO SKETCH

COMMENTS

Comments are ignored by the Arduino when it runs the sketch.

They are there for people reading the code: to explain what the program does, how it works, or why it's written the way it is.



```
/* Example of a sketch
This code turns on an LED on for one second,
then off for one second, and so on.
*/

void setup()
{
  pinMode(8, OUTPUT);
}

void loop()
{
  digitalWrite(8, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(8, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

WRITING BASIC ARDUINO PROGRAMS AND TESTING

ARDUINO SKETCH

FUNCTIONS

A *function* is a named piece of code. There are two special functions that are a part of every Arduino sketch: `setup()` and `loop()`.

`setup()` is called once, when the sketch starts. It's a good place to do setup tasks like setting pin modes.

`loop()` function is called over and over and is the heart of most sketches.

/ Example of a sketch*

This code turns on an LED on for one second, then off for one second, and so on.

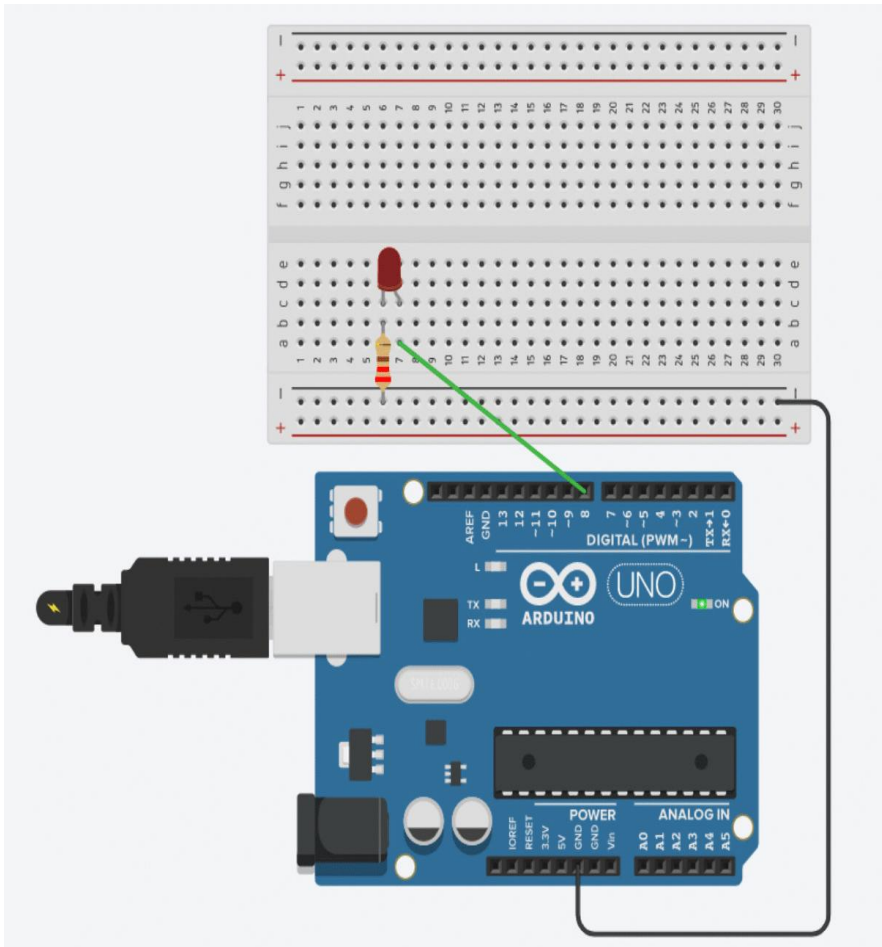
**/*

```
void setup()
{
  pinMode(8, OUTPUT);
}
```

```
void loop()
{
  digitalWrite(8, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(8, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

WRITING BASIC ARDUINO PROGRAMS AND TESTING

Example 2: LED Blink



```
/*
```

This code turns on an LED on for one second,
then off for one second, and so on.

```
*/
```

```
void setup()
```

```
{
```

```
  pinMode(8, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
  digitalWrite(8, HIGH);
```

```
  delay(1000); // Wait for 1000 millisecond(s)
```

```
  digitalWrite(8, LOW);
```

```
  delay(1000); // Wait for 1000 millisecond(s)
```

```
}
```

WRITING BASIC ARDUINO PROGRAMS AND TESTING

Example 2: LED Blink

VARIABLES

A variable is a place for storing a piece of data.

- It has a name, a type, and a value.
- For example,

```
int ledPin = 8;
int delayTime = 1000;
```

declares two variables of type integer, and assigns initial values to them.

```
/*
This code turns on an LED on for one second,
then off for one second, and so on.
*/

int ledPin = 8;
int delayTime = 1000;

void setup()
{
  pinMode(ledPin, OUTPUT);
}

void loop()
{
  digitalWrite(ledPin, HIGH);
  delay(delayTime); // Wait for delayTime millisecond(s)
  digitalWrite(ledPin, LOW);
  delay(delayTime); // Wait for delayTime millisecond(s)
}
```


WRITING BASIC ARDUINO PROGRAMS AND TESTING

SOME ARDUINO FUNCTIONS

- `pinMode(pin number, INPUT or OUTPUT)`
- `delay()` delay in milliseconds
- `delaymicrosecond()`
- `digitalRead(0-13)`
- `digitalWrite(pin number 0-13, LOW or HIGH)`
- `analogRead(A0-A5(0-1023))` (10 bits resolution)
- `analogWrite(pwmpins (3,5,6,9,10,11), value between (0-255))`
- `map(value, fromLow, fromHigh, toLow, toHigh)`

WRITING BASIC ARDUINO PROGRAMS AND TESTING

ASSIGNMENT 1: OPPOSITE BLINKING LIGHTS

Create a circuit called “LEDs_Blinking” with **two LEDs** that blink forever in an opposite way with some delay as follows:

Connections:

LED1 is connected to pin 2

LED2 is connected to pin 6

Functionality:

LED1 ON

LED2 OFF

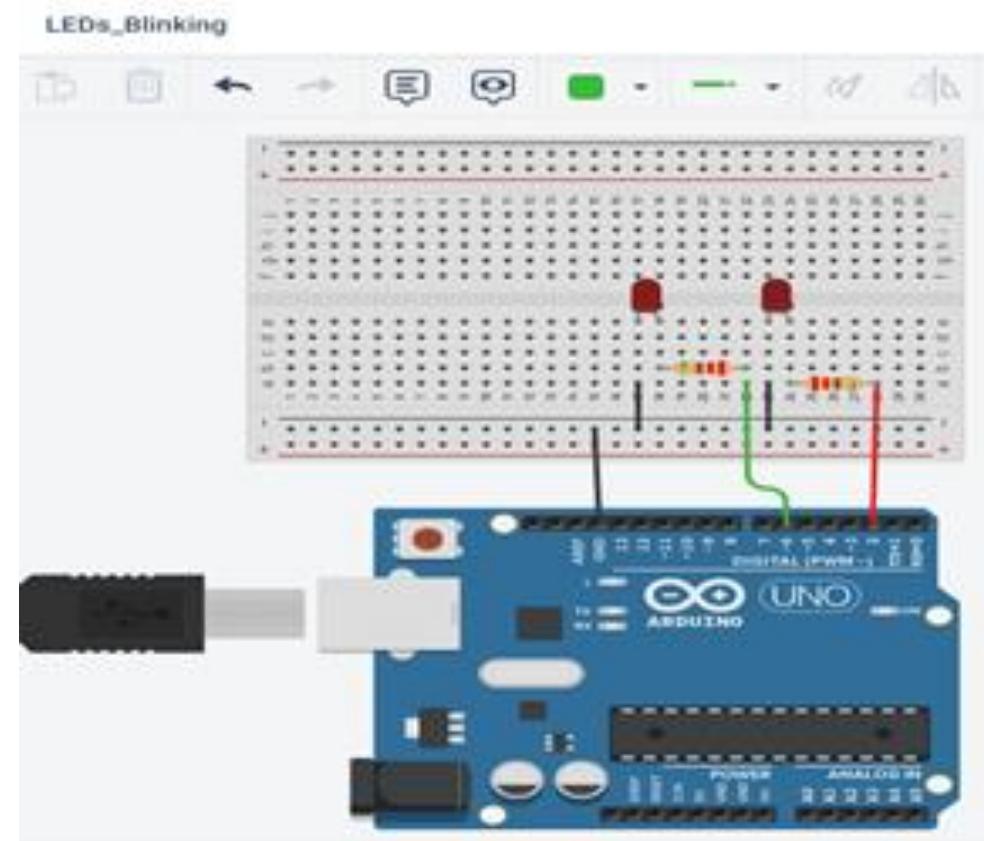
Delay 500 milliseconds

LED1 OFF

LED2 ON

Delay 500 milliseconds

Note: Add resistors of 220 Ohms to protect the LEDs.



WRITING BASIC ARDUINO PROGRAMS AND TESTING

ASSIGNMENT 2: LIGHT GAME

Create a circuit called “Light_Game” with **three LEDs** that blink in a continuous way in the following order:

LED1 ON, LED2 and LED3 OFF

DELAY 1 second

LED2 ON, LED1 and LED3 OFF

DELAY 1 second

LED3 ON, LED1 and LED2 OFF

DELAY 1 second

Notes:

Feel free to choose your pins

Add resistors of 220 Ohms to protect the LEDs

LEDs colors: LED1 red, LED2 green, LED3 yellow

